AtCoder風プラットフォーム 仕様書ドラフト v０．２

０．目的

- AtCoder風のオンラインジャッジ / 問題投稿 / コンテスト開催サービスを構築する。
- 可能な限り Heroku 中心で完結させる。
- 小規模ユーザー向けに、個人でも運用可能な構成で開始する。
- 将来的に、より安全なジャッジ基盤や機能拡張に対応できる構成にする。

１．前提・制約

- 最優先制約は Heroku 中心で完結すること。
- 初期ターゲットは小規模利用（個人運営 / 知人利用）。
- 収益化は行わず、無料のみ。
- 問題は誰でも作成可能、コンテストも誰でも開催可能。
- ジャッジ厳密度は「学習用より上、商用大規模OJ未満」。
- 初期対応問題種別は通常問題 + 部分点問題。
- インタラクティブ問題、出力のみ問題、リアクティブ採点、コード盗用検知は初期スコープ外。

２．スコープ

２．１　初期MVPに含める機能

- ユーザー登録 / ログイン
- 問題作成 / 編集 / 公開設定
- テストケース登録
- 提出 / 自動ジャッジ
- 提出履歴
- コンテスト作成 / 編集 / 公開
- コンテスト参加
- コンテスト順位表
- 解説投稿 / 公開
- 管理者による削除 / 非公開化 / 凍結

２．２　初期MVPに含めない機能

- インタラクティブ問題
- 出力のみ問題
- 問題ごとの複雑な custom checker
- リアルタイムランキング配信
- チーム戦
- 企業向け組織管理
- 有料プラン
- 高度な不正検知
- 複数認証プロバイダ連携

３．推奨技術方針

３．１　アプリケーション構成

- フロントエンド: Next.js
- API: Node.js (Next.js Route Handlers または Express)
- DB: PostgreSQL
- キュー / 一時状態: Redis
- 実行基盤: Worker プロセス + Docker 実行
- 永続ファイル: 外部オブジェクトストレージ

３．２　Heroku上のプロセス分離

- web dyno
- フロントエンド配信
- API処理
- 認証
- 参加 / 提出 / 問題管理
- worker dyno
- ジャッジジョブ取得
- コンパイル
- テストケース実行
- 採点結果保存

３．３　初期言語

- C++
- Python
- Java
- JavaScript

４．認証方針

４．１　初期採用

- GitHub ログインのみ

４．２　理由

- メールアドレス確認、パスワードリセット、アカウントロックなどの実装負債を避けられる。
- 競技プログラミング利用者と相性がよい。
- 初期の個人運営に向いている。

４．３　将来拡張

- Google ログイン追加
- メール + パスワード追加
- アカウント連携

５．ユーザーロール

５．１　ロール一覧

- Guest
- User
- Problem Author
- Contest Organizer
- Admin

５．２　権限概要

Guest

- 公開問題閲覧
- 公開コンテスト閲覧
- 解説閲覧（公開のみ）
- 順位表閲覧（公開のみ）

User

- ログイン
- 問題提出
- コンテスト参加
- 自分の提出履歴閲覧
- 自分のプロフィール編集

Problem Author

- 問題作成
- 自分の問題編集
- 自分の問題のテストケース編集
- 解説投稿
- 公開 / 非公開切替

Contest Organizer

- コンテスト作成
- コンテスト問題セット編集
- 開催時刻設定
- コンテスト公開 / 非公開切替
- コンテスト順位表管理

Admin

- 全問題 / 全コンテストの管理
- 不適切コンテンツの非公開化
- ユーザー凍結
- 再ジャッジ実行
- サービス全体設定変更

6. 公開範囲

6.1 問題

- public: 一覧に表示、誰でも閲覧 / 提出可能
- unlisted: URLを知るユーザーのみ閲覧可能
- private: 作成者と管理者のみ閲覧可能

6.2 コンテスト

- public: 一覧表示あり
- unlisted: URL共有型
- private: 招待 / 主催者限定

7. 問題仕様

7.1 問題基本情報

- タイトル
- スラッグ
- 制約文
- 問題文（Markdown）
- 入力形式
- 出力形式
- サンプル入出力
- タグ
- 難易度（任意）
- 公開設定
- 想定実行時間制限
- 想定メモリ制限
- 対応言語

7.2 採点方式

- AC / WA / TLE / MLE / RE / CE / IE
- 部分点対応
- 各テストケース or テストケースグループごとに点数配分可能
- 総得点はグループ得点の合計
- 通常問題は満点 or 0 点でも登録可能

7.3 問題データ編集方式

- Webフォームでメタデータを編集
- テストケースは ZIP アップロード
- 構成例:
- statement.md
- samples/sample1.in
- samples/sample1.out
- tests/group1/01.in

- tests/group 1 / 0 1 .out
- tests/group 2 / 0 1 .in
- tests/group 2 / 0 1 .out
- config.json

7.4 config.json 想定項目

- timeLimitMs
- memoryLimitMb
- scoringType
- groups
- name
- score
- tests
- languages
- checkerType (初期は exact のみ)

8. コンテスト仕様

8.1 基本情報

- タイトル
- スラッグ
- 説明
- 開始日時
- 終了日時
- タイムゾーン
- 公開範囲
- ペナルティ設定
- 順位表公開タイミング

8.2 問題セット

- コンテストに複数問題を紐付け
- 表示ラベル A, B, C, ...
- 各問題に配点を設定
- 問題ごとの公開時刻は初期MVPでは非対応

8.3 参加仕様

- 開始前は閲覧のみ
- 開催中は参加登録済みユーザーのみ提出可能
- 終了後の扱いは以下から設定可能
- 提出不可 / 閲覧のみ
- バーチャル参加可（将来拡張）

8.4 順位表

- 競プロ標準形式
- 表示項目

- 順位
- ユーザー名
- 総得点
- ペナルティ
- 各問題の提出状況
- 初期は AtCoder風の簡易集計
- 同点時はペナルティ → 最終更新時刻で順序決定

9. 提出・ジャッジ仕様

9.1 提出フロー

1. ユーザーがコード提出
2. APIが提出レコード作成
3. ソースコードを外部ストレージへ保存
4. Redisキューへジャッジジョブ投入
5. Workerが取得
6. 対応言語のコンパイル / 実行
7. テストケースを順次評価
8. 判定・得点・実行時間・メモリ使用量を保存
9. 順位表・提出一覧を更新

9.2 初期の言語別方針

- C++: g++
- Python: python 3
- Java: javac + java
- JavaScript: node

9.3 判定詳細

- CE: コンパイル失敗
- RE: 実行時例外 / 非 0 終了
- TLE: 制限時間超過
- MLE: メモリ制限超過
- WA: 出力不一致
- AC: 全テスト正解
- IE: ジャッジ基盤内部エラー

9.4 部分点仕様

- テストケースはグループ単位で採点
- グループ内全テスト通過でそのグループ得点を付与
- 1 件でも失敗するとそのグループ得点は 0
- 総得点 = 全グループ得点合計

9.5 実行順序

- 原則としてグループ順・ファイル順
- 早期終了モードを問題単位で設定可能

- ON: 致命的失敗で以降を省略
- OFF: 全件実行して詳細結果保存

９．６　checker

- 初期MVPでは exact checker のみ
- 前後空白 / 改行末尾の扱いは設定化可能
- custom checker は将来拡張

１０．画面一覧

１０．１　公開画面

- トップページ
- 問題一覧
- 問題詳細
- コンテスト一覧
- コンテスト詳細
- 順位表
- ユーザープロフィール

１０．２　認証後画面

- 提出ページ
- 提出結果一覧
- 自分の提出詳細
- 自分の問題一覧
- 問題作成 / 編集
- テストケースアップロード
- 解説編集
- 自分のコンテスト一覧
- コンテスト作成 / 編集

１０．３　管理画面

- ユーザー管理
- 問題モデレーション
- コンテストモデレーション
- 通報一覧
- 再ジャッジ管理

１１．データモデル（概要）

１１．１　Users

- id
- githubId
- username
- displayName

- avatarUrl
- bio
- role
- status
- createdAt
- updatedAt

11.2 Problems

- id
- authorId
- title
- slug
- statementMarkdown
- inputDescription
- outputDescription
- constraintsMarkdown
- visibility
- timeLimitMs
- memoryLimitMb
- supportedLanguages
- scoringType
- explanationMarkdown
- createdAt
- updatedAt

11.3 ProblemAssets

- id
- problemId
- kind
- storageKey
- checksum
- createdAt

11.4 ProblemTestGroups

- id
- problemId
- name
- score
- orderIndex

11.5 ProblemTestCases

- id
- groupId
- inputStorageKey
- outputStorageKey
- orderIndex

11.6 Contests

- id
- organizerId
- title
- slug
- descriptionMarkdown
- visibility
- startAt
- endAt
- penaltyMinutes
- scoreboardVisibility
- createdAt
- updatedAt

11.7 ContestProblems

- id
- contestId
- problemId
- label
- score
- orderIndex

11.8 ContestParticipants

- id
- contestId
- userId
- registeredAt

11.9 Submissions

- id
- userId
- problemId
- contestId nullable
- language
- sourceStorageKey
- status
- score
- totalTimeMs
- peakMemoryKb
- submittedAt
- judgedAt

11.10 SubmissionTestResults

- id
- submissionId

- groupId
- testCaseId
- verdict
- timeMs
- memoryKb
- message

１１.１１　Announcements / Reports

- id
- targetType
- targetId
- content
- status
- createdAt

１２.非機能要件

１２.１　想定規模

- 同時利用は小規模
- 数十〜数百ユーザー規模を想定
- 高頻度コンテスト同時開催は対象外

１２.２　性能要件

- 通常画面応答: 数秒以内
- 提出受付: 即時レスポンス
- 採点は非同期
- 順位表更新は eventual consistency を許容

１２.３　可用性

- Heroku障害時の完全継続は保証しない
- 個人運営向けのベストエフォート

１２.４　監査性

- 提出ログ
- 権限変更ログ
- 問題公開状態変更ログ
- 管理操作ログ

１３.セキュリティ方針

１３.１　重要前提

- 提出コードは信頼しない。
- 問題ZIPも信頼しない。
- Worker と Web は責務分離する。

１３.２　初期対策

- 提出サイズ上限
- 実行時間上限
- メモリ上限
- プロセス数制限
- ネットワーク無効
- 読み取り専用ルートFS
- 一時ディレクトリ分離
- 危険syscall制限（可能なら導入）

１３.３　モデレーション

- 問題 / 解説 / コンテスト説明の通報機能
- 管理者非公開化
- アカウント凍結

１４. Heroku適合方針

１４.１　Herokuで持つもの

- Webアプリ
- API
- DB
- Queue
- Worker

１４.２　Herokuローカルに置かないもの

- テストケース原本
- 提出ソース原本
- 添付ファイル
- ジャッジ生成物の長期保存

１４.３　初期実装方針

- 永続データは Postgres
- 一時ジョブは Redis
- ファイルはオブジェクトストレージ

１５. フェーズ分割

Phase　1

- GitHubログイン
- 問題投稿
- ZIP登録
- 提出
- 非同期ジャッジ
- 順位表

Phase 2

- 解説投稿
- unlisted / private
- 管理者モデレーション
- 再ジャッジ

Phase 3

- custom checker
- バーチャル参加
- 複数認証
- 組織向け権限

１６. 確定済み運用ポリシー

１６.１ 問題公開フロー

- 問題作成者は自分の問題を即時公開できる。
- 公開後に通報や管理者判断により非公開化できる。
- 初期MVPでは公開前承認フローは導入しない。

１６.２ コンテスト参加方式

- 開催中に初回提出した時点で自動参加とみなす。
- 任意で「参加」ボタンを押した場合も参加レコードを作成してよい。
- 順位表には参加レコードを持つユーザーのみ表示する。

１６.３ コンテスト終了後

- 終了後も通常提出を許可する。
- ただし、コンテスト順位表には終了後提出を反映しない。
- 終了後提出は通常提出扱いとして保存する。

１６.４ 順位表公開

- 順位表は開催中から常時公開する。
- 各参加者の各問題状態、得点、ペナルティを表示する。
- 初期MVPでは凍結順位表や部分凍結は導入しない。

１６.５ ユーザー名

- 初回ログイン時にサイト内表示名を設定する。
- GitHubアカウントは認証識別子としてのみ利用する。
- サイト内表示名は一意制約を持つ。
- 変更ポリシーは保留（初期MVPでは ３０ 日ごとなどの制限付き変更、または変更不可のどちらかを後続で確定）。

１６.６ 解説の公開タイミング

- 解説公開設定は問題単位またはコンテスト単位で持てる。

- 問題単位では、常時公開 / 非公開 / 日時指定公開を設定可能とする設計を推奨。
- コンテスト紐付け時は、コンテスト終了後に公開する設定を持てる。
- 初期MVPでは「常時公開」または「コンテスト終了後公開」の二択としてもよい。

１７. 詳細画面仕様

１７.１　トップページ

- 新着コンテスト
- おすすめ / 新着問題
- お知らせ
- ログイン導線

１７.２　問題一覧

- 検索
- タグ絞り込み
- 難易度絞り込み（任意）
- 公開状態に応じた表示制御
- 各問題の提出数 / AC数表示（任意）

１７.３　問題詳細

- 問題文
- 制約
- 入出力形式
- サンプル
- 解説リンク（公開条件を満たす場合のみ）
- 提出ボタン
- 提出状況（自分のみ）

１７.４　提出ページ

- 言語選択
- コードエディタ
- ソースサイズ表示
- 提出確認

１７.５　提出一覧

- フィルタ
- 自分のみ
- 問題別
- コンテスト別
- 判定別
- 言語別
- 表示項目
- 提出時刻
- ユーザー
- 問題

- 言語
- 判定
- 得点
- 実行時間
- メモリ

１７.６　提出詳細

- ソースコード
- 最終判定
- 各グループ結果
- 各テストケース結果（公開ポリシーによる）
- コンパイルエラー出力（本人 + 管理者のみ）

１７.７　問題作成 / 編集

- タイトル
- スラッグ
- 問題文Markdown
- 制約文
- 入出力形式
- 時間制限 / メモリ制限
- 対応言語
- 可視性
- 解説公開設定
- ZIPアップロード
- バージョン更新

１７.８　コンテスト一覧

- 開催予定
- 開催中
- 終了済み
- 主催者名
- 参加者数

１７.９　コンテスト詳細

- 概要
- 問題一覧
- 開始 / 終了
- 順位表リンク
- 参加状態

１７.１０　コンテスト編集

- 基本情報
- 問題セット
- 順位表公開設定
- 終了後提出の通常提出化設定

- 公開状態

１７.１１　初回ユーザー設定

- サイト内表示名
- 自己紹介（任意）
- アイコンは GitHub 由来を既定値として使用

１７.１２　管理画面

- ユーザー凍結
- 問題非公開化
- 解説非公開化
- コンテスト非公開化
- 再ジャッジ投入
- 通報処理

１８.権限マトリクス（主要操作）

操作	Guest	User	Problem Author	Contest Organizer	Admin
公開問題閲覧	可	可	可	可	可
非公開問題閲覧	不可	不可	自分のみ可	不可	可
問題作成	不可	可	可	可	可
自分の問題編集	不可	可	可	可	可
他人の問題編集	不可	不可	不可	不可	可
問題公開/非公開	不可	自分のみ可	自分のみ可	自分のみ可	可
公開問題に提出	不可	可	可	可	可
private問題に提出	不可	原則不可	作成者のみ可	原則不可	可
コンテスト作成	不可	可	可	可	可
自分のコンテスト編集	不可	可	可	可	可
他人のコンテスト編集	不可	不可	不可	不可	可

操作	Guest	User	Problem Author	Contest Organizer	Admin
コンテスト参加	不可	可	可	可	可
順位表閲覧	公開のみ可	可	可	可	可
解説投稿	不可	自分の問題のみ可	可	可	可
通報処理	不可	不可	不可	不可	可
再ジャッジ	不可	不可	自分の問題のみ可（任意）	自分のコンテスト紐付け問題のみ可（任意）	可

１９. コンテスト時間仕様

１９.１ 状態

- draft
- scheduled
- running
- ended
- cancelled

１９.２ 状態遷移

- draft -> scheduled
- scheduled -> running
- running -> ended
- draft -> cancelled
- scheduled -> cancelled

１９.３ 自動判定

- 現在時刻 < startAt: scheduled
- startAt <= 現在時刻 < endAt: running
- endAt <= 現在時刻: ended

１９.４ 提出ルール

- scheduled: 提出不可
- running: コンテスト提出可
- ended: コンテスト提出不可、通常提出は可

２０. 提出状態遷移

２０.１ Submission.status

- pending

- queued
- compiling
- running
- judging
- accepted
- wrong_answer
- time_limit_exceeded
- memory_limit_exceeded
- runtime_error
- compilation_error
- internal_error
- cancelled

２０．２　状態遷移

- pending -> queued
- queued -> compiling
- compiling -> running
- running -> judging
- judging -> accepted / wrong_answer / time_limit_exceeded / memory_limit_exceeded / runtime_error / internal_error
- compiling -> compilation_error
- queued / compiling / running / judging -> cancelled

２０．３　スコア確定

- 最終状態到達時に score / totalTimeMs / peakMemoryKb / judgedAt を確定する。
- コンテスト中提出で endAt 以前なら contest scoreboard に反映する。
- endAt 後提出は contest scoreboard に反映しない。

２１．順位表集計仕様

２１．１　問題単位集計

- 各問題について、その参加者のコンテスト中提出のうち最高得点を採用する。
- 満点到達前の失敗提出に対してペナルティを加算する。
- 初期MVPでは AtCoder風に「最終的に満点を取った問題について、満点到達前の失敗提出数 x penaltyMinutes」を採用する。

２１．２　総合集計

- totalScore = 各問題の採用得点の合計
- totalPenalty = 各問題ペナルティの合計 + 最終有効提出時刻ベースのタイム値
- 順位は totalScore desc, totalPenalty asc, lastAcceptedAt asc, userId asc の順で決定する。

２１．３　部分点問題

- 満点未達でも最高得点を採用する。
- ペナルティ加算方式は要件次第だが、初期MVPでは「満点到達時のみ失敗ペナルティ確定」に寄せると実装が簡潔。

２２. ZIP問題パッケージ仕様

２２.１ 必須ファイル

- statement.md
- config.json
- samples/*.in
- samples/*.out
- tests/<group-name>/*.in
- tests/<group-name>/*.out

２２.２ config.json 例の意味

- timeLimitMs: ミリ秒単位時間制限
- memoryLimitMb: MB単位メモリ制限
- scoringType: sum_of_groups
- groups: 配点と順序
- languages: 許可言語
- outputCompareMode: exact / ignore-trailing-spaces など

２２.３ バリデーション

- .in と .out は対で存在すること
- group score 合計はコンテスト問題配点と整合してもよいし、独立でもよい（初期MVPでは独立）
- サンプルと本番テストは分離する
- ZIP展開時の path traversal を拒否する
- 最大ファイル数、最大総容量、単一ファイル上限を設ける

２３. Heroku中心アーキテクチャ

２３.１ 推奨プロセス構成

- web: Next.js + API
- worker: judge queue consumer
- release: DB migration

２３.２ ストレージ責務

- Postgres: 正本データ
- Redis: キュー、一時ロック、短期キャッシュ
- Object Storage: ZIP、提出ソース、ログ成果物

２３.３ worker の責務

- 提出の取得
- 実行用一時ディレクトリ作成
- ソース展開
- コンパイル
- テストケース実行
- 結果保存

- 後処理削除

２３.４　セキュリティ実装の最低ライン

- ネットワーク遮断
- CPU / メモリ / 実行時間制限
- 一時ディレクトリ隔離
- ファイルサイズ制限
- fork 爆弾防止のプロセス上限
- 子プロセス kill と timeout cleanup

２４.追加で確定した運用ポリシー

２４.１　サイト内表示名の変更

- サイト内表示名は変更可能とする。
- 変更頻度は制限付きとし、初期MVPでは　３０　日ごとを推奨値とする。
- 表示名は一意でなければならない。
- 変更履歴は監査ログに保存する。

２４.２　他人の提出コードの閲覧

- 他人の提出コードは常に非公開とする。
- 提出コード本文を閲覧できるのは以下のみ。
- 提出者本人
- 管理者
- 問題作成者、コンテスト主催者であっても他人の提出コード本文は閲覧不可とする。

２４.３　提出詳細の公開範囲

- 他ユーザーに対しても提出詳細は公開可能とする。
- 公開対象には以下を含む。
- 最終判定
- 得点
- 実行時間
- メモリ使用量
- テストケース単位またはグループ単位の判定
- ただし、以下は本人と管理者のみ閲覧可能とする。
- 提出ソースコード全文
- コンパイルエラー全文
- 内部例外ログ

２４.４　問題作成 / 開催の制限

- 初期MVPでは、問題作成数およびコンテスト開催数に機能上の件数制限を設けない。
- ただし、サービス保護のため提出レート制限、ZIP容量制限、APIレート制限は別途設ける。
- 管理者は abuse 対応として個別ユーザーの作成権限停止を行える。

24.5 再ジャッジ権限

- 管理者は全再ジャッジが可能。
- 問題作成者は自分が作成した問題に対して再ジャッジを要求できる。
- コンテスト主催者は自分のコンテストに紐づく問題に対して再ジャッジを要求できる。
- 実際の再ジャッジ実行は非同期ジョブとして扱う。
- 監査ログに、要求者、対象、理由、件数、実行時刻を保存する。

25. DBスキーマ詳細（論理設計）

25.1 users

- id: uuid pk
- github_id: text unique not null
- username: text unique not null
- display_name: text not null
- avatar_url: text null
- bio: text null
- role: enum(user, admin) not null default user
- status: enum(active, frozen, deleted) not null default active
- username_changed_at: timestamptz null
- created_at: timestamptz not null
- updated_at: timestamptz not null

25.2 problems

- id: uuid pk
- author_id: uuid fk -> users.id not null
- title: text not null
- slug: text unique not null
- statement_markdown: text not null
- input_description_markdown: text not null
- output_description_markdown: text not null
- constraints_markdown: text null
- visibility: enum(public, unlisted, private) not null
- explanation_visibility: enum(always, contest_end, private) not null default private
- time_limit_ms: int not null
- memory_limit_mb: int not null
- scoring_type: enum(binary, sum_of_groups) not null
- supported_languages: jsonb not null
- latest_package_asset_id: uuid null fk -> problem_assets.id
- version: int not null default 1
- created_at: timestamptz not null
- updated_at: timestamptz not null

25.3 problem_assets

- id: uuid pk
- problem_id: uuid fk -> problems.id not null

- asset_kind: enum(package_zip, statement_attachment, checker_binary, submission_source, judge_log) not null
- storage_key: text not null
- file_name: text not null
- file_size: bigint not null
- checksum_sha 2 5 6 : text not null
- created_by: uuid fk -> users.id not null
- created_at: timestamptz not null

25.4 problem_test_groups

- id: uuid pk
- problem_id: uuid fk -> problems.id not null
- group_name: text not null
- score: int not null
- order_index: int not null
- created_at: timestamptz not null
- unique(problem_id, group_name)
- unique(problem_id, order_index)

25.5 problem_test_cases

- id: uuid pk
- group_id: uuid fk -> problem_test_groups.id not null
- case_name: text not null
- input_storage_key: text not null
- output_storage_key: text not null
- order_index: int not null
- created_at: timestamptz not null
- unique(group_id, case_name)
- unique(group_id, order_index)

25.6 problem_explanations

- id: uuid pk
- problem_id: uuid fk -> problems.id not null
- author_id: uuid fk -> users.id not null
- markdown: text not null
- visibility: enum(always, contest_end, private) not null
- published_at: timestamptz null
- created_at: timestamptz not null
- updated_at: timestamptz not null

25.7 contests

- id: uuid pk
- organizer_id: uuid fk -> users.id not null
- title: text not null
- slug: text unique not null
- description_markdown: text null

- visibility: enum(public, unlisted, private) not null
- start_at: timestamptz not null
- end_at: timestamptz not null
- penalty_minutes: int not null default 5
- scoreboard_visibility: enum(always) not null default always
- post_contest_submission_mode: enum(normal_submission_allowed) not null default normal_submission_allowed
- status_override: enum(none, cancelled) not null default none
- created_at: timestamptz not null
- updated_at: timestamptz not null

25.8 contest_problems

- id: uuid pk
- contest_id: uuid fk -> contests.id not null
- problem_id: uuid fk -> problems.id not null
- label: text not null
- score: int not null
- order_index: int not null
- explanation_visibility_override: enum(inherit, always, contest_end, private) not null default inherit
- created_at: timestamptz not null
- unique(contest_id, label)
- unique(contest_id, problem_id)
- unique(contest_id, order_index)

25.9 contest_participants

- id: uuid pk
- contest_id: uuid fk -> contests.id not null
- user_id: uuid fk -> users.id not null
- registered_at: timestamptz not null
- created_via: enum(auto_first_submission, manual_join) not null
- unique(contest_id, user_id)

25.10 submissions

- id: uuid pk
- user_id: uuid fk -> users.id not null
- problem_id: uuid fk -> problems.id not null
- contest_id: uuid null fk -> contests.id
- language: enum(cpp, python, java, javascript) not null
- source_asset_id: uuid fk -> problem_assets.id not null
- status: enum(pending, queued, compiling, running, judging, accepted, wrong_answer, time_limit_exceeded, memory_limit_exceeded, runtime_error, compilation_error, internal_error, cancelled) not null
- final_verdict: enum(AC, WA, TLE, MLE, RE, CE, IE, WJ) not null default WJ
- score: int not null default 0
- total_time_ms: int null

- peak_memory_kb: int null
- source_size_bytes: int not null
- judge_started_at: timestamptz null
- judged_at: timestamptz null
- submitted_at: timestamptz not null
- rejudge_of_submission_id: uuid null fk -> submissions.id

25.11 submission_group_results

- id: uuid pk
- submission_id: uuid fk -> submissions.id not null
- group_id: uuid fk -> problem_test_groups.id not null
- verdict: enum(AC, WA, TLE, MLE, RE, CE, IE, SKIP) not null
- score_awarded: int not null default 0
- max_time_ms: int null
- max_memory_kb: int null
- created_at: timestamptz not null
- unique(submission_id, group_id)

25.12 submission_test_results

- id: uuid pk
- submission_id: uuid fk -> submissions.id not null
- test_case_id: uuid fk -> problem_test_cases.id not null
- verdict: enum(AC, WA, TLE, MLE, RE, IE, SKIP) not null
- time_ms: int null
- memory_kb: int null
- message: text null
- created_at: timestamptz not null
- unique(submission_id, test_case_id)

25.13 scoreboard_rows

- id: uuid pk
- contest_id: uuid fk -> contests.id not null
- user_id: uuid fk -> users.id not null
- total_score: int not null default 0
- total_penalty: int not null default 0
- last_accepted_at: timestamptz null
- last_submission_at: timestamptz null
- details_json: jsonb not null
- updated_at: timestamptz not null
- unique(contest_id, user_id)

25.14 reports

- id: uuid pk
- reporter_id: uuid fk -> users.id not null
- target_type: enum(problem, contest, explanation, user) not null
- target_id: uuid not null

- reason: text not null
- status: enum(open, reviewing, resolved, rejected) not null default open
- created_at: timestamptz not null
- updated_at: timestamptz not null

25.15 audit_logs

- id: uuid pk
- actor_user_id: uuid null fk -> users.id
- action_type: text not null
- target_type: text not null
- target_id: text not null
- metadata_json: jsonb not null
- created_at: timestamptz not null

26. API仕様（初期版）

26.1 認証

- GET /api/auth/login/github
- GET /api/auth/callback/github
- POST /api/auth/logout
- GET /api/me
- PATCH /api/me/profile

26.2 問題

- GET /api/problems
- POST /api/problems
- GET /api/problems/:problemId
- PATCH /api/problems/:problemId
- POST /api/problems/:problemId/publish
- POST /api/problems/:problemId/unpublish
- POST /api/problems/:problemId/package
- GET /api/problems/:problemId/explanation
- PUT /api/problems/:problemId/explanation

26.3 コンテスト

- GET /api/contests
- POST /api/contests
- GET /api/contests/:contestId
- PATCH /api/contests/:contestId
- POST /api/contests/:contestId/publish
- POST /api/contests/:contestId/unpublish
- POST /api/contests/:contestId/join
- GET /api/contests/:contestId/scoreboard
- POST /api/contests/:contestId/problems
- DELETE /api/contests/:contestId/problems/:contestProblemId

２６.４　提出

- POST /api/submissions
- GET /api/submissions
- GET /api/submissions/:submissionId
- POST /api/submissions/:submissionId/rejudge

２６.５　管理

- GET /api/admin/reports
- POST /api/admin/reports/:reportId/resolve
- POST /api/admin/users/:userId/freeze
- POST /api/admin/users/:userId/unfreeze
- POST /api/admin/problems/:problemId/hide
- POST /api/admin/contests/:contestId/hide

２６.６　API共通仕様

- 認証必須APIはセッションベースで保護する。
- 書き込み系APIは CSRF または SameSite cookie 前提の保護を行う。
- 一覧APIは page / limit / cursor のいずれかでページングする。
- エラー形式は { code, message, details? } を統一採用する。

２７.非同期ジョブ仕様

２７.１　ジョブ種別

- submission.judge
- submission.rejudge
- scoreboard.rebuild
- package.validate
- package.extract
- cleanup.temp_files

２７.２　submission.judge

入力: - submissionId - requestedAt - reason(normal | rejudge)

出力: - submissions 更新 - submission_group_results 作成 - submission_test_results 作成 - scoreboard_rows 更新（必要時） - audit_logs 追加

２７.３　package.validate

- ZIP整合性確認
- path traversal 検出
- 許可拡張子確認
- 必須ファイル確認
- config.json schema 検証
- group / score / case 対応確認

27.4 scoreboard.rebuild

- contestId を受け取る
- contest中有効提出のみで再集計する
- scoreboard_rows を全再生成する
- 大量再ジャッジ後や不整合修復に使用する

28. ジャッジ実行仕様

28.1 共通前処理

1. submission レコードを lock する。
2. status を queued -> compiling に進める。
3. object storage から提出ソースを取得する。
4. object storage から問題パッケージを取得する。
5. 一時ディレクトリに展開する。

28.2 言語別コンパイル / 実行

- C++
- compile: g++ Main.cpp -O2 -std=gnu++17 -o main
- run: ./main
- Python
- compile step なし
- run: python3 main.py
- Java
- compile: javac Main.java
- run: java Main
- JavaScript
- compile step なし
- run: node main.js

28.3 実行制御

- 各テストケースごとに timeout を設定する。
- メモリ上限を設定する。
- 標準入力に .in を与える。
- 標準出力をキャプチャする。
- 標準エラーはログとして保持するが公開範囲を制限する。

28.4 採点

- compare mode は初期MVPでは exact または ignore-trailing-spaces を許容する設計とする。
- group 内すべて AC で group score を付与する。
- group 内に 1 件でも失敗があれば group score は 0 。
- total score は group score 合計。
- final verdict は次の優先順で決定する。
- CE
- IE
- RE

- MLE
- TLE
- WA
- AC

２８.５　後処理

- judged_at を設定する。
- scoreboard 対象なら差分更新または rebuild を行う。
- 一時ファイルを削除する。
- 失敗時も cleanup.temp_files を投入する。

２９.追加で確定した制限・実行ポリシー

２９.１　提出レート制限

- 提出レート制限は二層で設ける。
- 問題単位クールダウン
- ユーザー全体単位の時間窓上限制限
- 初期推奨値(devcenter.heroku.com)
- 再ジャッジ要求は別枠で制限する。
- 同一ユーザー:　１　分あたり　３　要求まで
- 同一対象問題への連続再ジャッジは　６０　秒クールダウン
- レート超過時は HTTP　４２９　を返し、再試行可能時刻を明示する。

２９.２　ZIPアップロード上限

- 上限は三層で設ける。
- ZIP総容量上限
- 単一ファイル上限
- 総ファイル数上限
- 初期推奨値
- ZIP総容量:　６４　MB
- 単一ファイル:　８　MB
- 総ファイル数:　１０００
- 追加制限
- サンプルファイル数上限
- テストケース総数上限
- 展開後総容量上限
- ZIPアップロード時は展開前・展開後の両方で検証する。

２９.３　テストケース詳細の公開

- テストケース詳細の公開粒度は問題ごとに設定可能とする。
- 設定候補
- group_only: グループ単位のみ公開
- case_index_only: ケース番号のみ公開
- case_name_visible: ケース名も公開
- 初期既定値は case_index_only を推奨する。

- 作問意図や hidden test 名称漏えい防止の観点から、case_name_visible は任意明示設定時のみ許可する。

２９.４　コンパイル/実行環境バージョン固定

- 実行環境は「メジャーバージョン固定」を基本方針とする。
- パッチバージョンはセキュリティ更新・不具合修正のため範囲内で追従する。
- アプリ側では major version alias を設定して、再現性と保守性のバランスを取る。
- 問題ごとに異なる実行環境は初期MVPでは提供しない。

２９.５　ジャッジ比較方式

- 比較方式は問題ごとに設定可能とする。
- 初期対応モード
- exact
- ignore_trailing_spaces
- 将来拡張候補
- tokenized_whitespace
- floating_point_tolerance
- custom_checker

３０.Heroku運用方針（２０２６時点の前提）

３０.１　スタック方針

- 新規アプリは Heroku-２４ を前提とする。
- Heroku-２４ は Ubuntu ２４.０４ LTS ベースで、２０２９ 年 ４ 月までサポートされる。
- 旧 stack 依存設計は避ける。

３０.２　配備方式

- web プロセスは buildpack ベースを第一候補とする。
- judge worker は必要に応じて container stack を使う。
- ただし container stack は curated OS layer を含まないため、ベースイメージ保守責任が利用者側にある。
- 可能な限り buildpack で済む部分は buildpack を優先する。

３０.３　ファイル保存方針

- dyno ローカルファイルシステムは永続保存先として使用しない。
- テストケース ZIP、提出コード、ログ成果物は object storage に保存する。
- 一時展開物のみ dyno ローカルに置き、ジョブ終了時に削除する。

３０.４　ランタイム固定の実務方針

- Node.js は package.json の engines で major range 指定を行う。
- Python は .python-version で major version を明示する。
- サポート切れメジャーは build 時に問題になるため、定期見直しを保守作業に含める。
- judge 用 Java / g++ / Node / Python バージョンは release note に明記して変更履歴を残す。

３１. 推奨初期ランタイム方針

３１.１ Webアプリ

- Node.js ２４.x を推奨
- 理由: Active LTS であり、Herokuでも本番利用推奨帯に入るため

３１.２ Judge worker 内言語

- C++: g++ １３ 系相当以上を想定
- Python: ３.１３ 系を推奨
- Java: ２１ LTS 系を推奨
- JavaScript: Node.js ２４.x を推奨
- ただし judge worker が container stack の場合、上記は自前Docker imageで固定し、変更時は明示的な judge environment version を上げる。

３１.３ judge_environment_version

- submissions は採点時の judge_environment_version を保存する。
- 再ジャッジ時は新旧環境の差分を追跡できるようにする。
- 問題ページに現行実行環境バージョンを表示する。

３２. 最終アーキテクチャ確定

３２.１ judge worker の配備方式

- web プロセスは buildpack ベースで配備する。
- judge worker は container ベースで配備する。
- 採用理由:
- web は Heroku curated stack の恩恵を受けやすく保守が軽い。
- judge worker は C++ / Python / Java / JavaScript の実行環境を Docker image で明示固定しやすい。

３２.２ オブジェクトストレージ方針

- アプリケーションコード上は抽象化層を設け、 storage provider に依存しない設計とする。
- 初期インターフェース:
- putObject(key, stream, metadata)
- getObject(key)
- deleteObject(key)
- createSignedUploadUrl(params)
- createSignedDownloadUrl(params)
- 実装候補:
- S３ 互換ストレージ
- Heroku アドオン経由のストレージ
- ただし、小規模MVPでは一部データを Postgres へ代替保存可能とする。

３２．３　DB 代替保存ポリシー

Postgres に保存してよいもの

- 提出ソースコード全文（text）
- コンパイルエラー全文（text）
- 小さい judge ログ（text）
- 問題文 Markdown（text）
- 解説 Markdown（text）
- 小さな ZIP（bytea） ※上限を厳格に絞る場合のみ

Postgres 保存を非推奨とするもの

- 大きい問題 ZIP 一式
- hidden test の大量入出力ファイル
- 大量再ジャッジで繰り返し参照される大容量バイナリ
- 添付画像や将来のメディアファイル

初期MVP推奨

- 提出ソースコード: Postgres 保存で開始してよい
- 問題 ZIP: まずは object storage 推奨
- どうしても DB 代替したい場合は、問題 ZIP も Postgres bytea 保存で始めてもよいが、総容量上限・件数上限を強く絞る

３２．４　セッション方式

- セッションは DB セッション方式を採用する。
- セッションテーブルを Postgres に持つ。
- サーバーサイドセッション ID は httpOnly / secure / sameSite cookie で保持する。
- Redis をセッションの正本には使わない。

３２．５　フロント / API 構成

- Next.js 単体構成を採用する。
- UI、SSR/ISR、API Route / Route Handler を単一アプリ内で持つ。
- 管理画面も同一アプリ内に含める。

３２．６　ランキング更新方式

- 基本は提出ごとの差分更新を行う。
- 不整合検知時または大量再ジャッジ時には scoreboard.rebuild を実行する。
- 差分更新に失敗した場合でも、再構築で自己修復できる設計とする。

３２．７　初期MVPの管理機能

- 通報処理
- ユーザー凍結 / 凍結解除
- 問題非公開化
- コンテスト非公開化
- 解説非公開化

- 再ジャッジ要求 / 実行管理

３３. DB代替とストレージの最終判断指針

３３.１　代替判断ルール

- text で扱える小容量データは Postgres を優先してよい。
- 複数ファイル / 大容量 / 頻繁なダウンロードを伴うものは object storage を優先する。
- 永続ファイルを dyno ローカルには置かない。

３３.２　このプロジェクト向けの現実解

- MVP　１:
- 問題文 / 解説 / 提出ソース / CE ログは Postgres
- 問題 ZIP / テストケース群は object storage
- MVP　１.５（極限簡略版）:
- 問題 ZIP も Postgres bytea に保存
- ただし問題総数・ZIP総容量・テストケース数をかなり小さく制限
- 将来:
- すべてのファイル系は object storage に統一

３３.３　移行容易性

- problems.latest_package_asset_id と submissions.source_asset_id は、storage_key ベースで参照する抽象設計にする。
- 初期実装で DB 保存していても、後から object storage へ移行できるよう asset kind / storage backend / checksum を保持する。

３４. 最終固定値と実装決定

３４.１　提出レート制限の最終確定

- 同一ユーザー・同一問題への通常提出クールダウン:　１０　秒
- 同一ユーザーの通常提出総数上限:　１　分あたり　２０　件
- 同一ユーザーの再ジャッジ要求上限:　１　分あたり　３　件
- 同一対象問題への再ジャッジクールダウン:　６０　秒
- 管理者は緊急対応時に上限を一時的にバイパス可能

３４.２　問題 ZIP の保存先

- 本番初期MVPから問題 ZIP は object storage に保存する。
- 開発環境または緊急簡易構成に限り、DB bytea backend を選択可能とする。
- テストケース群も object storage を正本とする。

３４.３　judge worker container の固定

- judge worker は versioned container image（例: judge-env:v１）で配備する。
- ベース OS は Ubuntu　２４.０４　系を採用する。
- 初期ランタイム固定値:
- g++　１３　系

- Python　３．１３　系
- Java　２１　LTS 系
- Node.js　２４.x
- judge_environment_version は `v1` から開始し、ランタイム変更時に `v2`, `v3` と更新する。

３４．４　case_name_visible のUI表記

- 問題編集画面で以下の補足文を表示する。
- 「ケース名を公開すると hidden test の意図や構成が推測されやすくなります。通常はケース番号のみ公開を推奨します。」
- 初期既定値は case_index_only とする。

３４．５　再ジャッジ要求理由

- 再ジャッジ要求時は理由入力を必須とする。
- 理由は audit_logs に保存する。
- UIではプリセット選択 + 自由記述を許可する。
- 例: `judge update`, `testcase fix`, `scoring bug`, `manual review`

３５．仕様書ｖ１　結論

- 本システムは、Heroku 中心で動作する小規模向け AtCoder 風 OJ / Contest プラットフォームである。
- web は buildpack、judge worker は container とし、永続ファイルは dyno ローカルに保存しない。
- 認証は GitHub ログイン、セッションは DB セッション、フロント/API は Next.js 単体構成とする。
- 問題は誰でも作成・即公開でき、コンテストも誰でも開催できる。
- 対応問題種別は通常問題 + 部分点問題、対応言語は C++ / Python / Java / JavaScript とする。
- 順位表は開催中から常時公開し、差分更新 + 再構築で整合性を保つ。
- 他人の提出コードは非公開だが、判定・得点・時間・メモリ・ケース/グループ結果は公開できる。
- 問題 ZIP / テストケースは object storage を正本とし、提出ソースや小さなテキストログは Postgres 保存で開始可能とする。
- 管理者・問題作成者・コンテスト主催者は権限に応じて再ジャッジを要求できる。
- 初期MVPでも、通報、凍結、非公開化、再ジャッジ管理を備える。

以上を本プロジェクトの初版仕様書として確定する。

３６．実装用ディレクトリ構成（推奨）

```
repo/
├── apps/
│   └── web/
│       ├── app/
│       │   ├── (public)/
│       │   │   ├── page.tsx
│       │   │   ├── problems/
│       │   │   ├── contests/
│       │   │   └── users/
│       │   ├── (auth)/
```

```
│      │   │   ├── login/
│      │   │   └── onboarding/
│      │   ├── (dashboard)/
│      │   │   ├── problems/
│      │   │   ├── contests/
│      │   │   ├── submissions/
│      │   │   └── admin/
│      │   └── api/
│      │       ├── auth/
│      │       ├── me/
│      │       ├── problems/
│      │       ├── contests/
│      │       ├── submissions/
│      │       └── admin/
│      ├── components/
│      ├── features/
│      │   ├── auth/
│      │   ├── problems/
│      │   ├── contests/
│      │   ├── submissions/
│      │   ├── scoreboard/
│      │   └── admin/
│      ├── lib/
│      │   ├── db/
│      │   ├── auth/
│      │   ├── storage/
│      │   ├── queue/
│      │   ├── rate-limit/
│      │   ├── markdown/
│      │   └── judge/
│      ├── prisma/
│      ├── public/
│      ├── tests/
│      ├── package.json
│      └── Procfile
├── workers/
│   └── judge/
│      ├── src/
│      │   ├── index.ts
│      │   ├── jobs/
│      │   ├── runners/
│      │   │   ├── cpp/
│      │   │   ├── python/
│      │   │   ├── java/
│      │   │   └── javascript/
│      │   ├── sandbox/
│      │   ├── storage/
│      │   ├── compare/
```

```
│       │   ├── scoreboard/
│       │   └── cleanup/
│       ├── Dockerfile
│       ├── package.json
│       └── tsconfig.json
├── packages/
│   ├── shared/
│   │   ├── src/
│   │   │   ├── types/
│   │   │   ├── constants/
│   │   │   ├── schemas/
│   │   │   └── utils/
│   │   └── package.json
│   └── ui/
│       ├── src/
│       └── package.json
├── infra/
│   ├── heroku/
│   │   ├── app.json
│   │   ├── web.env.example
│   │   └── worker.env.example
│   └── scripts/
│       ├── release.sh
│       ├── migrate.sh
│       └── seed.sh
├── docs/
│   ├── api/
│   ├── database/
│   ├── judge/
│   └── operations/
├── .github/
│   └── workflows/
├── package.json
├── pnpm-workspace.yaml
└── README.md
```

37. 実装フェーズ分解

Phase 0: 基盤初期化

目的: - モノレポ初期化 - Heroku 配備枠組み作成 - DB / Redis / session / auth の土台構築

タスク: - pnpm workspace 構成作成 - Next.js アプリ初期化 - judge worker 初期化 - Prisma 導入
接続設定 - Redis 接続設定 - GitHub OAuth 設定 - DB session 実装 - 共通 env schema 実装 - 監査ログ基盤追加

完了条件: - ローカルで web 起動可能 - Heroku review / staging に web 配備可能 - GitHub ログ
回オンボーディングで表示名登録可能

Phase 1 : 問題管理MVP

目的: - 問題作成、編集、公開、ZIP登録まで動作させる

タスク: - problems / problem_assets / problem_test_groups / problem_test_cases モデル作成 - 問題画面 - 問題編集画面 - Markdown レンダラ導入 - ZIPアップロード API - package.validate ジョブ実装 package.extract ジョブ実装 - 問題一覧 / 詳細ページ実装 - 可視性制御（public/unlisted/private）実装 - 解説モデル実装 - 解説公開タイミング制御実装

完了条件: - 問題を作成し ZIP を登録できる - バリデーション失敗時に理由が返る - public/unlisted/private で表示制御が正しい

Phase 2 : 提出・ジャッジMVP

目的: - 通常提出から採点完了までの一連フローを完成させる

タスク: - submissions / submission_group_results / submission_test_results 作成 - 提出API実装 - 提出ト制限実装 - 提出一覧 / 提出詳細画面実装 - judge worker queue consumer 実装 - C++ runner 実 runner 実装 - Java runner 実装 - JavaScript runner 実装 - exact compare 実装 - ignore_trai compare 実装 - 部分点計算実装 - final verdict 決定ロジック実装 - source code の DB 保存実装 cleanup 実装

完了条件: - 4 言語で提出できる - AC/WA/TLE/MLE/RE/CE/IE が反映される - 部分点が正しく計算される - 他人のソースコードが閲覧不可である

Phase 3 : コンテストMVP

目的: - コンテスト作成・参加・順位表まで完成させる

タスク: - contests / contest_problems / contest_participants / scoreboard_rows 作成 - コンテスト作成 - コンテスト編集画面 - 問題セット管理 UI - scheduled / running / ended 状態判定実装 - 初提出時自 ジック実装 - コンテスト提出判定実装 - 終了後通常提出化ロジック実装 - 差分順位表更新実装 - scoreboard.rebuild 実装 - 順位表画面実装 - コンテスト詳細画面実装

完了条件: - running 中のみコンテスト提出が集計対象になる - ended 後の提出は順位表に反映されない - 順位表が差分更新される - 不整合時に再構築で復旧できる

Phase 4 : 管理・運用MVP

目的: - 最低限の運営機能を実装する

タスク: - reports モデル実装 - 通報 UI / API 実装 - 管理画面実装 - ユーザー凍結 / 凍結解除実装 スト / 解説非公開化実装 - 再ジャッジ要求UI実装 - 再ジャッジ監査ログ実装 - 表示名変更制限実装 - case visibility 設定 UI 実装

完了条件: - 不適切コンテンツを管理者が止められる - 再ジャッジ要求が非同期で処理される - 表示名変更制限が機能する

Phase 5：本番化・保守性強化

目的: - staging / production 運用と障害復旧性を高める

タスク: - health check 実装 - structured logging 実装 - Sentry 等の例外監視導入 - DB migration storage backend 抽象差し替え確認 - judge_environment_version 表示 - rate limit 管理UIまたは設定ファイル整備 - バックアップ / 復旧手順作成 - runbook 作成

完了条件: - 障害時の確認手順がドキュメント化されている - 再デプロイと migration が安定している - staging から production への移行手順が明確

38. 実装優先順位（着手順）

1. auth + session + user onboarding
2. DB schema 初版
3. 問題作成 / 問題一覧 / 問題詳細
4. ZIP validation / extraction
5. 提出API + worker queue
6. 1 言語（Python）だけでジャッジ通す
7. 残り 3 言語追加
8. 部分点
9. コンテスト
10. 順位表差分更新
11. 管理画面
12. 再ジャッジ
13. 運用監視

補足: - 最初から 4 言語同時対応を目指すとデバッグが重い。 - 最初は Python 1言語で end-to-end を通し、その後 runner を横展開する方が強い。

39. API実装順

39.1 最優先

- GET /api/me
- PATCH /api/me/profile
- POST /api/problems
- GET /api/problems
- GET /api/problems/:problemId
- PATCH /api/problems/:problemId
- POST /api/problems/:problemId/package
- POST /api/submissions
- GET /api/submissions/:submissionId

39.2 次点

- POST /api/contests
- PATCH /api/contests/:contestId

- GET /api/contests/:contestId
- GET /api/contests/:contestId/scoreboard
- POST /api/submissions/:submissionId/rejudge

39.3 運用系

- GET /api/admin/reports
- POST /api/admin/users/:userId/freeze
- POST /api/admin/problems/:problemId/hide
- POST /api/admin/contests/:contestId/hide

40. DB migration順

1. users
2. audit_logs
3. problems
4. problem_assets
5. problem_test_groups
6. problem_test_cases
7. problem_explanations
8. contests
9. contest_problems
10. contest_participants
11. submissions
12. submission_group_results
13. submission_test_results
14. scoreboard_rows
15. reports

41. 受け入れテスト（MVP）

41.1 認証

- GitHub ログインできる
- 初回のみ表示名登録画面へ遷移する
- 表示名重複時は登録できない

41.2 問題

- 作成者が問題を作れる
- ZIP が壊れていると拒否される
- private 問題は他人に見えない
- unlisted 問題は一覧に出ないが URL 直打ちで見える

41.3 提出

- public 問題に提出できる
- 10 秒クールダウン中は提出できない
- Python/C++/Java/JavaScript で採点できる

- 他人の提出コードを見られない
- ケース/グループ結果は公開設定に従う

41.4 コンテスト

- running 中の提出だけ順位表対象になる
- 初提出で参加レコードが作成される
- ended 後の提出は通常提出扱いで保存される
- 順位表は開催中から閲覧できる

41.5 管理

- 通報を作成できる
- 管理者が問題を非公開化できる
- 管理者がユーザーを凍結できる
- 問題作成者または主催者が再ジャッジ要求できる

42. Codex CLI に渡す実装指示の単位

42.1 1チケットを小さく切る

悪い例: - 「AtCoderみたいなサイトを全部作って」

良い例: - 「Prisma で users/problems/problem_assets/problem_test_groups/problem_test_cases の schema を作成し、初回 migration まで出して」 - 「Next.js app router で /problems/new と /proble [id]/edit を作り、public/unlisted/private を扱える form を実装して」 - 「judge worker の package.validat ジョブを TypeScript で実装し、ZIP の必須ファイル検証と path traversal 拒否まで入れて」 - 「Python ラン ナーだけ先に実装し、stdin/stdout 比較で AC/WA/TLE/RE を返すようにして」

42.2 先に固定してから依頼するもの

- DB schema
- env 名
- queue payload schema
- storage interface
- verdict enum
- visibility enum

42.3 後から差し替えやすく保つもの

- storage backend 実装
- compare mode 実装詳細
- judge environment version
- logging/monitoring 基盤

43. 次に作るべき成果物

1. Prisma schema 全量
2. app/web の初期 directory と package.json

3. judge worker の Dockerfile と package.json
4. storage interface 定義
5. queue payload schema 定義
6. ZIP package schema（config.json のJSON Schema）
7. 提出レート制限 middleware
8. Python runner の最小実装

44. この仕様書を使った次工程

- 工程 1 : 実装タスク一覧化
- 工程 2 : DB schema 全量作成
- 工程 3 : web 初期雛形作成
- 工程 4 : worker 初期雛形作成
- 工程 5 : Python 1 言語でE 2 E疎通
- 工程 6 : 4 言語化
- 工程 7 : contest / scoreboard / admin を追加